

**Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)**

Department of Computer Science and Engineering
II Year M.Tech Integrated CSE (III Semester)

Academic Year 2024-25

ICS1312 – Java Programming Lab

Name: Moogambigai A
Reg No: 312237001027

Name: Joice Anancia
Reg No: 312237001020

1. Problem Statement: Student Grading System

Motivation for the Problem:

Traditional grading and record-keeping methods are often labor-intensive, prone to human error, and can lead to inconsistencies. Additionally, as educational institutions grow, managing records manually becomes increasingly challenging, especially when it involves multiple roles like students, faculty, and administrators. An automated system not only reduces the workload on faculty and administrative staff but also ensures secure, role-based access, enhancing data privacy and user experience. By streamlining tasks like grade entry, report generation, and attendance tracking, this system supports academic accountability and provides students with timely feedback on their performance.

Problem Description:

The aim is to design and develop a student grading system that simplifies managing student records while ensuring role-based access. This system should provide students, faculty, and administrators with access to relevant data, ensuring secure user interactions. It also improve administrative efficiency, reduce manual errors, and enhance academic transparency. The focus will be on creating a modular system with object-oriented principles, file-based data storage, and an interactive interface for a user-friendly experience.

Inputs:

Outputs:

Scope and Limitations:

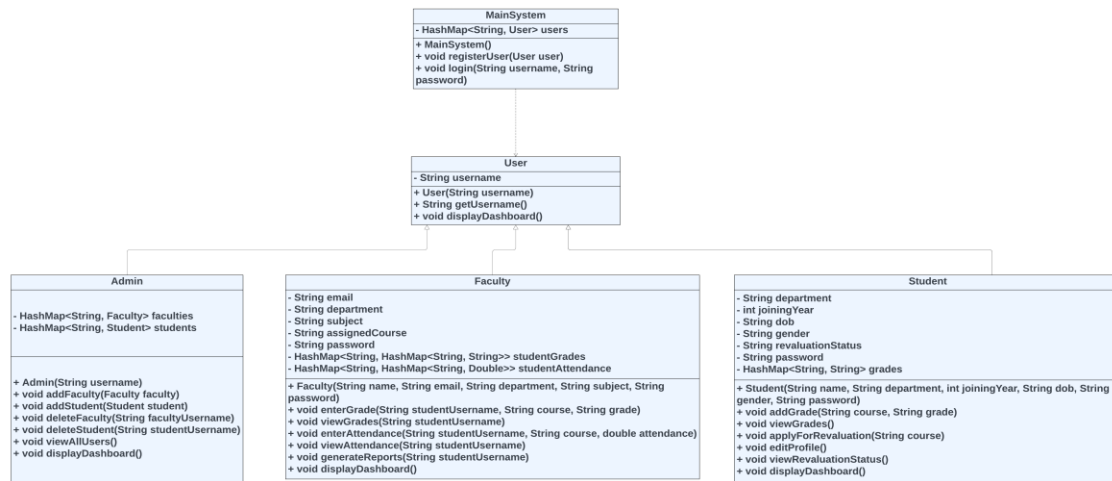
Scope:

- Provides role-based access for secure data handling.
- Focuses on faculty, student, and admin functionalities related to grading and data validation.
- Supports institutional email validation and file-based data storage, ensuring data integrity.

Limitations:

- Restricted to users within the institution, validated through email addresses with the @ssn.edu.in suffix.
- Limited to grading and basic administrative functions without integration with external educational databases or other systems.

2.Design of the Solution:



3. Design Alternative

For an enhanced design of the student grading system, integrating Java Database Connectivity (**JDBC**) and a graphical user interface (**GUI**) could provide more dynamic data management and a user-friendly experience. With JDBC, the system could directly interact with a relational database, allowing for real-time data storage and retrieval, which would improve efficiency compared to file-based storage. This setup would streamline the processes of adding, deleting, and updating records for students and faculty, while the GUI would present a visually accessible platform for users, enhancing ease of use. A GUI also simplifies user input, reducing errors and improving validation with intuitive fields for email, grade entries, and login credentials. This combined approach would support scalability, enabling smoother access and data management, especially useful for larger educational institutions.

4. Object-Oriented Concepts Used

5. Modules Split-Up:

6. Implementation of the Solution:

Class User

```
package com.sgs.studentgradingsystem.model;
```

```
public abstract class User {  
    protected String name;  
    protected String email;  
    protected String password;  
  
    public User(String name, String email, String password) {  
        this.name = name;  
        this.email = email;  
        this.password = password;  
    }  
  
    public String getEmail() { return email; }  
    public String getPassword() { return password; }  
    public String getName() { return name; }  
  
    public abstract void displayDashboard();  
}
```

Class Admin

```
package com.sgs.studentgradingsystem.model;
```

```
public class Admin extends User {  
    private static final String ADMIN_EMAIL = "admin@ssn.edu.in";  
    private static final String ADMIN_PASSWORD = "Admin@1234";  
  
    public Admin() {  
        super("Admin", ADMIN_EMAIL, ADMIN_PASSWORD);  
    }  
}
```

```

@Override
public void displayDashboard() {
    System.out.println("===== ADMIN DASHBOARD =====");
    System.out.println("1. Add Faculty");
    System.out.println("2. Delete Faculty");
    System.out.println("3. Add Student");
    System.out.println("4. Delete Student");
    System.out.println("6. View All Students");
    System.out.println("7. View All Faculty");
    System.out.println("8. Exit Admin Dashboard");
}

public boolean isValidUsername(String username) {
    return ADMIN_EMAIL.equalsIgnoreCase(username);
}

public boolean isValidPassword(String password) {
    return ADMIN_PASSWORD.equals(password);
}
}

```

Class Faculty

```

package com.sgs.studentgradingsystem.model;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

public class Faculty extends User {
    private String department;
    private String subject;
    private List<Student> studentsList = new ArrayList<>();
    private HashMap<Student, HashMap<String, Integer>> studentCredits = new HashMap<>(); //
    Stores credits by student and course

    public Faculty(String name, String email, String department, String subject, String password) {
        super(name, email, password);
        this.department = department;
        this.subject = subject;
        this.studentsList = studentsList;
    }

    @Override
    public void displayDashboard() {
        System.out.println("===== FACULTY DASHBOARD =====");
        System.out.println("1. View Profile");
    }
}

```

```

        System.out.println("2. Set Grades for Students");
        System.out.println("3. View Students' Grades");
        System.out.println("4. Review Revaluation Requests");
        System.out.println("5. Calculate CGPA for a Student");
        System.out.println("6. Exit");
    }

    // Method to set grades for students
    public void setGrades(Scanner sc, Student student) {
        System.out.print("Enter course name: ");
        String course = sc.nextLine();

        System.out.print("Enter grade (O, A+, A, B+, B, C, U): ");
        String grade = sc.nextLine();

        System.out.print("Enter credit for course: ");
        int credit = sc.nextInt();
        sc.nextLine(); // Consume newline

        setGrade(student, course, grade, credit);
        System.out.println("Grade set successfully for " + student.getName());
    }

    public void viewStudentGrades(Student student) {
        System.out.println("Grades for " + student.getName() + ":");
        for (Map.Entry<String, String> entry : student.getGrades().entrySet()) {
            System.out.println("Course: " + entry.getKey() + ", Grade: " + entry.getValue());
        }
    }

    public void reviewRevaluationRequests(Scanner sc) {
        System.out.print("Enter student's email: ");
        String studentEmail = sc.nextLine().trim();

        Student student = findStudentByEmail(studentEmail);
        if (student != null) {
            HashMap<String, RevaluationRequest> requests = student.getRevaluationRequests();
            if (requests.isEmpty()) {
                System.out.println("No revaluation requests found for " + student.getName());
            } else {
                for (RevaluationRequest request : requests.values()) {
                    System.out.println("Revaluation request for course: " + request.getCourse());
                    System.out.println("Current Grade: " + request.getCurrentGrade());
                }
            }
        } else {
            System.out.println("Student not found.");
        }
    }
}

```

```

private Student findStudentByEmail(String studentEmail) {
    for (Student student : studentsList) { // Ensure studentsList is accessible
        if (student.getEmail().equalsIgnoreCase(studentEmail)) {
            return student;
        }
    }
    return null; // Return null if not found
}

public void setGrade(Student student, String course, String grade, int credit) {
    student.addGrade(course, grade);
    studentCredits.computeIfAbsent(student, k -> new HashMap<>()).put(course, credit);
}

public double calculateStudentCGPA(Student student) {
    HashMap<String, String> grades = student.getGrades();
    HashMap<String, Integer> credits = studentCredits.getDefault(student, new
HashMap<>());

    int totalCredits = 0;
    int totalPoints = 0;

    for (String course : grades.keySet()) {
        int gradePoint = getGradePoint(grades.get(course));
        int courseCredit = credits.getDefault(course, 0);

        totalPoints += gradePoint * courseCredit;
        totalCredits += courseCredit;
    }

    return totalCredits == 0 ? 0 : (double) totalPoints / totalCredits;
}

private int getGradePoint(String grade) {
    switch (grade) {
        case "O": return 10;
        case "A+": return 9;
        case "A": return 8;
        case "B+": return 7;
        case "B": return 6;
        case "C": return 5;
        case "U": return 0;
        default: return 0;
    }
}

public String getSubject() {
    return this.subject;
}

```

```

    public String getEmail() {
        return email;
    }

    public String toFileFormat() {
        return name + "," + email + "," + department + "," + subject + "," + password;
    }

    @Override
    public String toString() {
        return "Faculty Name: " + getName() +
            ", Email: " + getEmail() +
            ", Department: " + department +
            ", Subject: " + subject;
    }
}

```

Class Student

```

package com.sgs.studentgradingsystem.model;

```

```

import java.util.HashMap;

```

```

public class Student extends User {
    private String department;
    private int joiningYear;
    private String dob;
    private String gender;
    private HashMap<String, String> grades = new HashMap<>();
    private HashMap<String, RevaluationRequest> revaluationRequests = new HashMap<>();

    public Student(String name, String email, String department, int joiningYear, String dob, String
gender, String password) {
        super(name, email, password);
        this.department = department;
        this.joiningYear = joiningYear;
        this.dob = dob;
        this.gender = gender;
    }

    @Override
    public void displayDashboard() {
        System.out.println("===== STUDENT DASHBOARD =====");
        System.out.println("1. View Profile");
        System.out.println("2. View Grades");
        System.out.println("3. Apply for Revaluation");
        System.out.println("4. Exit");
    }

    public void addGrade(String course, String grade) {
        grades.put(course, grade);
    }
}

```

```

    }

    public HashMap<String, String> getGrades() {
        return grades;
    }

    public HashMap<String, RevaluationRequest> getRevaluationRequests() {
        return revaluationRequests;
    }

    public void applyForRevaluation(String course, String facultyName, int currentGrade) {
        if (grades.containsKey(course) && grades.get(course).equalsIgnoreCase("U")) {
            RevaluationRequest request = new RevaluationRequest(getName(), course, facultyName,
currentGrade);
            revaluationRequests.put(course, request);
            System.out.println("Revaluation request submitted for course: " + course);
        } else {
            System.out.println("Revaluation can only be applied for courses with a 'U' grade.");
        }
    }

    private int getGradePoint(String grade) {
        switch (grade) {
            case "O": return 10;
            case "A+": return 9;
            case "A": return 8;
            case "B+": return 7;
            case "B": return 6;
            case "C": return 5;
            case "U": return 0;
            default: return -1;
        }
    }

    public void updateGrade(String course, int newGrade) {
        if (revaluationRequests.containsKey(course)) {
            grades.put(course, String.valueOf(newGrade));
            RevaluationRequest request = revaluationRequests.get(course);
            request.setNewGrade(newGrade);
            System.out.println("Grade updated to: " + newGrade + " for course: " + course);
        } else {
            System.out.println("No revaluation request found for course: " + course);
        }
    }

    public void markRevaluationReviewed(String course) {
        if (revaluationRequests.containsKey(course)) {
            RevaluationRequest request = revaluationRequests.get(course);
            request.setNewGrade(request.getNewGrade());
            System.out.println("Revaluation request for " + course + " marked as reviewed.");
        } else {

```



```

        System.out.println("No revaluation request found for course: " + course);
    }
}

public void approveRevaluation(String course) {
    RevaluationRequest request = revaluationRequests.get(course);
    if (request != null) {
        request.setNewGrade(request.getNewGrade());
        System.out.println("Revaluation approved for " + course);
    } else {
        System.out.println("No revaluation request found for " + course);
    }
}

public String getEmail() {
    return email;
}

public String toFileFormat() {
    return name + "," + email + "," + department + "," + joiningYear + "," + dob + "," + gender +
    "," + password;
}

@Override
public String toString() {
    return "Student Name: " + getName() +
        ", Email: " + getEmail() +
        ", Department: " + department +
        ", Joining Year: " + joiningYear +
        ", DOB: " + dob +
        ", Gender: " + gender;
}
}

```

Class RevaluationRequest

package com.sgs.studentgradingsystem.model;

```

public class RevaluationRequest {
    private static int requestCounter = 0;
    private int requestId;
    private String studentName;
    private String courseName;
    private String facultyName;
    private int currentGrade;
    private int newGrade;

    public RevaluationRequest(String studentName, String courseName, String facultyName, int
currentGrade) {
        this.requestId = ++requestCounter;
        this.studentName = studentName;
    }
}

```

```

        this.courseName = courseName;
        this.facultyName = facultyName;
        this.currentGrade = currentGrade;
        this.newGrade = currentGrade;
    }

    public int getRequestId() { return requestId; }
    public String getStudentName() { return studentName; }
    public String getCourseName() { return courseName; }
    public String getFacultyName() { return facultyName; }
    public int getCurrentGrade() { return currentGrade; }
    public int getNewGrade() { return newGrade; }

    public void setNewGrade(int newGrade2) {
        this.newGrade = newGrade2;
    }
    public String getCourse() {
        return courseName;
    }
}

```

Class InputValidator

```

package com.sgs.studentgradingsystem.utils;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.Scanner;

public class InputValidator {
    public static String validateStringInput(Scanner sc, String prompt) {
        while (true) {
            System.out.print(prompt);
            String input = sc.nextLine();
            if (input.matches("[a-zA-Z ]+")) {
                return input;
            } else {
                System.out.println("Invalid input. Please enter only letters and spaces.");
            }
        }
    }

    public static int validateIntegerInput(Scanner sc, String prompt, int min, int max) {
        while (true) {
            System.out.print(prompt);
            if (sc.hasNextInt()) {
                int input = sc.nextInt();
                sc.nextLine(); // Consume newline
                if (input >= min && input <= max) {
                    return input;
                }
            }
        }
    }
}

```

```

        } else {
            System.out.println("Invalid input. Please enter a number between " + min + " and " +
max + ".");
        }
    } else {
        System.out.println("Invalid input. Please enter a valid number.");
        sc.next();
    }
}

// Validate email input
public static String validateEmailInput(Scanner sc, String prompt) {
    while (true) {
        System.out.print(prompt);
        String input = sc.nextLine();
        if (input.matches("^[\\w.%+-]+@ssn\\.edu\\.in$")) {
            return input;
        } else {
            System.out.println("Invalid input. Please enter a valid SSN email address (ending with
@ssn.edu.in).");
        }
    }
}

public static String validatePasswordInput(Scanner sc, String prompt) {
    while (true) {
        System.out.print(prompt);
        String input = sc.nextLine();
        if (input.length() >= 8 &&
            input.matches(".*[A-Z].*") &&
            input.matches(".*[a-z].*") &&
            input.matches(".*[!@#$%^&*()_.?\\':{}|<>].*")) {
            return input;
        } else {
            System.out.println("Invalid input. Password must be at least 8 characters long, contain at
least one uppercase letter, one lowercase letter, and one special character.");
        }
    }
}

public static String validateDOBInput(Scanner sc, String prompt) {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");
    while (true) {
        System.out.print(prompt);
        String input = sc.nextLine();
        if (isValidDOB(input, formatter)) {
            return input;
        } else {
            System.out.println("Invalid input. Please enter a valid date of birth in the format dd-MM-
yyyy and ensure it's in the past.");
        }
    }
}

```

```

    }

    private static boolean isValidDOB(String dob, DateTimeFormatter formatter) {
        try {
            LocalDate date = LocalDate.parse(dob, formatter);
            return date.isBefore(LocalDate.now());
        } catch (DateTimeParseException e) {
            return false;
        }
    }
}

```

Class Logger

```

package com.sgs.studentgradingsystem.utils;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Logger {
    public static void logLoginDetails(String role, String email) {
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
        LocalDateTime now = LocalDateTime.now();
        String logEntry = dtf.format(now) + " - " + role + " login - Email: " + email + "\n";

        try (BufferedWriter writer = new BufferedWriter(new FileWriter("login_details.log", true))) {
            writer.write(logEntry);
        } catch (IOException e) {
            System.out.println("An error occurred while logging login details: " + e.getMessage());
        }
    }
}

```

Class MainSystem

```

package com.sgs.studentgradingsystem.main;

import com.sgs.studentgradingsystem.model.*;
import com.sgs.studentgradingsystem.utils.InputValidator;
import java.util.ArrayList;
import java.util.List;
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;

```

```

import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

class UnauthorizedAccessException extends Exception {
    public UnauthorizedAccessException(String message) {
        super(message);
    }
}

public class MainSystem {
    private Map<String, Faculty> faculties = new HashMap<>();
    private Map<String, Student> students = new HashMap<>();
    private List<Student> studentsList = new ArrayList<>();
    private Admin admin = new Admin();

    public MainSystem() {
        loadFacultiesFromFile();
        loadStudentsFromFile();
    }

    public void addFaculty(Faculty faculty) {
        faculties.put(faculty.getEmail(), faculty);
        saveFacultiesToFile();
        System.out.println("Faculty added successfully!");
    }

    public void addStudent(Student student) {
        students.put(student.getEmail(), student);
        saveStudentsToFile();
        System.out.println("Student added successfully!");
    }

    public void viewAllFaculties() {
        System.out.println("===== Faculty List =====");
        faculties.values().forEach(System.out::println);
    }

    public void viewAllStudents() {
        System.out.println("===== Students List =====");
        students.values().forEach(System.out::println);
    }

    private void loadFacultiesFromFile() {
        try (BufferedReader reader = new BufferedReader(new FileReader("faculty.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] data = line.split(",");
                Faculty faculty = new Faculty(data[0], data[1], data[2], data[3], data[4]);
                faculties.put(faculty.getEmail(), faculty);
            }
        }
    }
}

```

```

    }
    } catch (IOException e) {
        System.out.println("Error loading faculties from file.");
    }
}

private void saveFacultiesToFile() {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter("faculty.txt"))) {
        for (Faculty faculty : faculties.values()) {
            writer.write(faculty.toFileFormat());
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error saving faculties to file.");
    }
}

private void loadStudentsFromFile() {
    try (BufferedReader reader = new BufferedReader(new FileReader("student.txt"))) {
        String line;
        while ((line = reader.readLine()) != null) {
            String[] data = line.split(",");
            Student student = new Student(data[0], data[1], data[2], Integer.parseInt(data[3]), data[4],
data[5], data[6]);
            students.put(student.getEmail(), student);
            studentsList.add(student);
        }
    } catch (IOException e) {
        System.out.println("Error loading students from file.");
    }
}

private void saveStudentsToFile() {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter("student.txt"))) {
        for (Student student : students.values()) {
            writer.write(student.toFileFormat());
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error saving students to file.");
    }
}

public static void main(String[] args) {
    MainSystem system = new MainSystem();
    Scanner sc = new Scanner(System.in);
    boolean exit = false;

    System.out.println("===== WELCOME TO THE STUDENT GRADING SYSTEM =====");

    while (!exit) {

```

```

        System.out.println("===== MAIN MENU =====");
        System.out.println("1. Login as Admin");
        System.out.println("2. Login as Faculty");
        System.out.println("3. Login as Student");
        System.out.println("4. Exit");
        System.out.print("Enter your choice: ");

        int choice = sc.nextInt();
        sc.nextLine();

        switch (choice) {
            case 1:
                system.adminLogin(sc);
                break;
            case 2:
                system.facultyLogin(sc);
                break;
            case 3:
                system.studentLogin(sc);
                break;
            case 4:
                exit = true;
                System.out.println("Exiting the system!");
                break;
            default:
                System.out.println("Invalid choice. Please try again.");
        }
    }
    sc.close();
}

private void adminLogin(Scanner sc) {
    System.out.print("Enter Admin Username: ");
    String username = sc.nextLine();

    System.out.print("Enter Admin Password: ");
    String password = sc.nextLine();

    if (admin.isValidUsername(username) && admin.isValidPassword(password)) {
        System.out.println("Login successful!");
        logLoginDetails("Admin", username);
        adminOperations(sc);
    } else {
        System.out.println("Invalid credentials.");
    }
}

private void facultyLogin(Scanner sc) {
    String email = InputValidator.validateEmailInput(sc, "Enter Faculty Email: ");
    String password = InputValidator.validatePasswordInput(sc, "Enter Faculty Password: ");

```

```

Faculty faculty = faculties.get(email);
if (faculty != null && faculty.getPassword().equals(password)) {
    System.out.println("Login successful!");
    logLoginDetails("Faculty", email);
    facultyOperations(sc, faculty);
} else {
    System.out.println("Invalid credentials.");
}
}

private void studentLogin(Scanner sc) {
    String email = InputValidator.validateEmailInput(sc, "Enter Student Email: ");
    String password = InputValidator.validatePasswordInput(sc, "Enter Student Password: ");

    Student student = students.get(email);
    if (student != null && student.getPassword().equals(password)) {
        System.out.println("Login successful!");
        logLoginDetails("Student", email);
        studentOperations(sc, student);
    } else {
        System.out.println("Invalid credentials.");
    }
}

private void adminOperations(Scanner sc) {
    boolean loop = true;
    while (loop) {
        System.out.println("===== ADMIN DASHBOARD =====");
        System.out.println("1. Add Faculty");
        System.out.println("2. Delete Faculty");
        System.out.println("3. Add Student");
        System.out.println("4. Delete Student");
        System.out.println("5. View All Faculties");
        System.out.println("6. View All Students");
        System.out.println("7. Exit");

        int choice = sc.nextInt();
        sc.nextLine();

        switch (choice) {
            case 1:
                addFaculty(new Faculty(
                    InputValidator.validateStringInput(sc, "Enter Faculty Name: "),
                    InputValidator.validateEmailInput(sc, "Enter Faculty Email: "),
                    InputValidator.validateStringInput(sc, "Enter Faculty Department: "),
                    InputValidator.validateStringInput(sc, "Enter Faculty Subject: "),
                    InputValidator.validatePasswordInput(sc, "Set a Password: ")
                ));
                break;
            case 2:
                String emailToDelete = InputValidator.validateEmailInput(sc, "Enter Faculty Email to

```



```

delete: ");
    faculties.remove(emailToDelete);
    System.out.println("Faculty deleted successfully.");
    break;
case 3:
    addStudent(new Student(
        InputValidator.validateStringInput(sc, "Enter Student Name: "),
        InputValidator.validateEmailInput(sc, "Enter Student Email: "),
        InputValidator.validateStringInput(sc, "Enter Student Department: "),
        InputValidator.validateIntegerInput(sc, "Enter Student Joining Year: ", 1900, 2100),
        InputValidator.validateDOBInput(sc, "Enter Student Date of Birth (DOB): "),
        InputValidator.validateStringInput(sc, "Enter Student Gender: "),
        InputValidator.validatePasswordInput(sc, "Set a Password: ")
    ));
    break;
case 4:
    String studentEmailToDelete = InputValidator.validateEmailInput(sc, "Enter Student
Email to delete: ");
    students.remove(studentEmailToDelete);
    System.out.println("Student deleted successfully.");
    break;
case 5:
    viewAllFaculties();
    break;
case 6:
    viewAllStudents();
    break;
case 7:
    loop = false;
    break;
default:
    System.out.println("Invalid option. Please try again.");
    break;
    }
}
}

```

```

private void facultyOperations(Scanner sc, Faculty faculty) {
    boolean loop = true;
    while (loop) {
        System.out.println("===== FACULTY DASHBOARD =====");
        System.out.println("1. View Profile");
        System.out.println("2. Set Grades for Students");
        System.out.println("3. View Students' Grades");
        System.out.println("4. Review Revaluation Requests");
        System.out.println("5. Exit");

        int choice = sc.nextInt();
        sc.nextLine();

        switch (choice) {

```

```

        case 1:
            System.out.println("Profile: " + faculty);
            break;
        case 2:
            setGrades(sc, faculty);
            break;
        case 3:
            viewStudentGrades(sc, faculty);
            break;
        case 4:
            reviewRevaluationRequests(sc);
            break;
        case 5:
            loop = false;
            break;
        default:
            System.out.println("Invalid option. Please try again.");
            break;
    }
}
}

private void studentOperations(Scanner sc, Student student) {
    boolean loop = true;
    while (loop) {
        System.out.println("===== STUDENT DASHBOARD =====");
        System.out.println("1. View Profile");
        System.out.println("2. View Grades");
        System.out.println("3. Apply for Revaluation");
        System.out.println("4. Exit");

        int choice = sc.nextInt();
        sc.nextLine();

        switch (choice) {
            case 1:
                System.out.println("Profile: " + student);
                break;
            case 2:
                viewGrades(student);
                break;
            case 3:
                applyForRevaluation(sc, student);
                break;
            case 4:
                loop = false;
                break;
            default:
                System.out.println("Invalid option. Please try again.");
                break;
        }
    }
}

```

```

    }
}

private void setGrades(Scanner sc, Faculty faculty) {
    System.out.print("Enter student's email: ");
    String studentEmail = sc.nextLine().trim();
    Student student = students.get(studentEmail);

    if (student != null) {
        faculty.setGrades(sc, student);
    } else {
        System.out.println("Student not found.");
    }
}

private void viewStudentGrades(Scanner sc, Faculty faculty) {
    System.out.print("Enter student's email: ");
    String studentEmail = sc.nextLine().trim();
    Student student = findStudentByEmail(studentEmail); // Use email for finding student

    if (student != null) {
        faculty.viewStudentGrades(student); // Pass the found student to faculty method
    } else {
        System.out.println("Student not found.");
    }
}

// Method to find student by email in the student database or list
private Student findStudentByEmail(String email) {
    for (Student student : studentsList) { // Ensure studentsList is accessible
        if (student.getEmail().equalsIgnoreCase(email)) {
            return student;
        }
    }
    return null; // Return null if not found
}

public void reviewRevaluationRequests(Scanner sc) {
    System.out.print("Enter student's email: ");
    String studentEmail = sc.nextLine().trim();

    Student student = findStudentByEmail(studentEmail);
    if (student != null) {
        HashMap<String, RevaluationRequest> requests = student.getRevaluationRequests();
        if (requests.isEmpty()) {
            System.out.println("No revaluation requests found for " + student.getName());
        } else {
            for (RevaluationRequest request : requests.values()) {
                System.out.println("Revaluation request for course: " + request.getCourse());
            }
        }
    }
}

```

```

        System.out.println("Current Grade: " + request.getCurrentGrade());
    }
} else {
    System.out.println("Student not found.");
}
}

private Student findStudentByName(String studentName) {
    for (Student student : studentsList) {
        if (student.getName().equalsIgnoreCase(studentName)) {
            return student;
        }
    }
    return null;
}

private static void viewGrades(Student student) {
    HashMap<String, String> grades = student.getGrades();
    System.out.println("Grades for " + student.getName() + ":\n");
    grades.forEach((course, grade) -> System.out.println(course + ": " + grade));

    try (BufferedWriter writer = new BufferedWriter(new FileWriter("grades_report.txt", true))) {
        writer.write("Grades for " + student.getName() + ":\n");
        grades.forEach((course, grade) -> {
            try {
                writer.write(course + ": " + grade + "\n");
            } catch (IOException e) {
                System.out.println("Error writing grade to file for course: " + course + " - " +
e.getMessage());
            }
        });
        writer.write("\n");
        System.out.println("Grades report updated in the file.");
    } catch (IOException e) {
        System.out.println("Error writing grades to file: " + e.getMessage());
    }
}

private static void applyForRevaluation(Scanner sc, Student student) {
    String course = InputValidator.validateStringInput(sc, "Enter the course for revaluation: ");
    student.applyForRevaluation(course, course, 0);
}

private void logLoginDetails(String userType, String identifier) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter("login_history.txt", true))) {
        String dateTime = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
    }
}

```

```

        writer.write(userType + " " + identifier + " logged in at " + dateTime);
        writer.newLine();
    } catch (IOException e) {
        System.out.println("Error logging login details.");
    }
}
}
}

```

7. Evaluation and Test Cases

```

===== WELCOME TO THE STUDENT GRADING SYSTEM =====
===== MAIN MENU =====
1. Login as Admin
2. Login as Faculty
3. Login as Student
4. Exit
Enter your choice: 1
Enter Admin Username: admin@ssn.edu.in
Enter Admin Password: Admin@1234
Login successful!
===== ADMIN DASHBOARD =====
1. Add Faculty
2. Delete Faculty
3. Add Student
4. Delete Student
5. View All Faculties
6. View All Students
7. Exit
5
===== Faculty List =====
Faculty Name: raj, Email: raj@ssn.edu.in, Department: cse, Subject: python
Faculty Name: Shakthi, Email: shakthi@ssn.edu.in, Department: cse, Subject: DPSD
Faculty Name: jack, Email: jack@ssn.edu.in, Department: cse, Subject: C

```

```

Faculty Name: raji, Email: raji@ssn.edu.in, Department: cse, Subject: java
Faculty Name: sree, Email: sree@ssn.edu.in, Department: cse, Subject: os
Faculty Name: spin, Email: spin@ssn.edu.in, Department: cse, Subject: os
===== ADMIN DASHBOARD =====
1. Add Faculty
2. Delete Faculty
3. Add Student
4. Delete Student
5. View All Faculties
6. View All Students
7. Exit
6
===== Students List =====
Student Name: moogss, Email: moogss@ssn.edu.in, Department: cse, Joining Year: 2023,
DOB: 30-09-2005, Gender: f
Student Name: anjana, Email: anjana@ssn.edu.in, Department: cse, Joining Year: 2023,
DOB: 27-08-2005, Gender: f
Student Name: dileep, Email: dileep@ssn.edu.in, Department: it, Joining Year: 2023,
DOB: 22-12-2004, Gender: m
Student Name: Nandy, Email: nandy@ssn.edu.in, Department: cse, Joining Year: 2023,
DOB: 04-11-2005, Gender: f
Student Name: joice, Email: joice@ssn.edu.in, Department: cse, Joining Year: 2023,
DOB: 22-12-2005, Gender: f
===== ADMIN DASHBOARD =====
1. Add Faculty
2. Delete Faculty
3. Add Student
4. Delete Student
5. View All Faculties
6. View All Students
7. Exit
2
Enter Faculty Email to delete: sree@ssn.edu.in
Faculty deleted successfully.
===== ADMIN DASHBOARD =====
1. Add Faculty
2. Delete Faculty
3. Add Student
4. Delete Student
5. View All Faculties
6. View All Students
7. Exit
5
===== Faculty List =====
Faculty Name: raj, Email: raj@ssn.edu.in, Department: cse, Subject: python
Faculty Name: Shakthi, Email: shakthi@ssn.edu.in, Department: cse, Subject: DPSD
Faculty Name: jack, Email: jack@ssn.edu.in, Department: cse, Subject: C

```

Faculty Name: raji, Email: raji@ssn.edu.in, Department: cse, Subject: java

Faculty Name: spin, Email: spin@ssn.edu.in, Department: cse, Subject: os

===== ADMIN DASHBOARD =====

1. Add Faculty
2. Delete Faculty
3. Add Student
4. Delete Student
5. View All Faculties
6. View All Students
7. Exit

4

Enter Student Email to delete: dileep@ssn.edu.in

Student deleted successfully.

===== ADMIN DASHBOARD =====

1. Add Faculty
2. Delete Faculty
3. Add Student
4. Delete Student
5. View All Faculties
6. View All Students
7. Exit

6

===== Students List =====

Student Name: moogss, Email: moogss@ssn.edu.in, Department: cse, Joining Year: 2023,
DOB: 30-09-2005, Gender: f

Student Name: anjana, Email: anjana@ssn.edu.in, Department: cse, Joining Year: 2023,
DOB: 27-08-2005, Gender: f

Student Name: Nandy, Email: nandy@ssn.edu.in, Department: cse, Joining Year: 2023,
DOB: 04-11-2005, Gender: f

Student Name: joice, Email: joice@ssn.edu.in, Department: cse, Joining Year: 2023,
DOB: 22-12-2005, Gender: f

===== ADMIN DASHBOARD =====

1. Add Faculty
2. Delete Faculty
3. Add Student
4. Delete Student
5. View All Faculties
6. View All Students
7. Exit

7

===== MAIN MENU =====

1. Login as Admin
2. Login as Faculty
3. Login as Student
4. Exit

```
Enter your choice: 2
Enter Faculty Email: shakthi@ssn.edu.in
Enter Faculty Password: Shakthi@1234
Login successful!
===== FACULTY DASHBOARD =====
1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit
2
Enter student's email: anjana@ssn.edu.in
Enter course name: DPSD
Enter grade (O, A+, A, B+, B, C, U): O
Enter credit for course: 4
Grade set successfully for anjana
===== FACULTY DASHBOARD =====
1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit|
3
```



```

Enter student's email: anjana@ssn.edu.in
Grades for anjana:
Course: DPSD, Grade: O
===== FACULTY DASHBOARD =====
1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit
5
===== MAIN MENU =====
1. Login as Admin
2. Login as Faculty
3. Login as Student
4. Exit
Enter your choice: 3
Enter Student Email: anjana@ssn.edu.in
Enter Student Password: Anjana@1234
Login successful!
===== STUDENT DASHBOARD =====
1. View Profile
2. View Grades
3. Apply for Revaluation
4. Exit
1
Profile: Student Name: anjana, Email: anjana@ssn.edu.in, Department: cse, Joining
Year: 2023, DOB: 27-08-2005, Gender: f
===== STUDENT DASHBOARD =====
1. View Profile
2. View Grades
3. Apply for Revaluation
4. Exit
2
Grades for anjana:
DPSD: O
Grades report updated in the file.
===== STUDENT DASHBOARD =====
1. View Profile
2. View Grades
3. Apply for Revaluation
4. Exit
3
Enter the course for revaluation: DPSD
Revaluation can only be applied for courses with a 'U' grade.
===== STUDENT DASHBOARD =====
1. View Profile
2. View Grades

```

```
3. Apply for Revaluation
4. Exit
4
===== MAIN MENU =====
1. Login as Admin
2. Login as Faculty
3. Login as Student
4. Exit
Enter your choice: 2
Enter Faculty Email: spin@ssn.edu.in
Enter Faculty Password: Spin@1234
Login successful!
===== FACULTY DASHBOARD =====
1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit
2
Enter student's email: nandy@ssn.edu.in
Enter course name: os
Enter grade (O, A+, A, B+, B, C, U): U
Enter credit for course: 3
Grade set successfully for Nandy
```

===== FACULTY DASHBOARD =====

1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit

3

Enter student's email: nandy@ssn.edu.in

Grades for Nandy:

Course: os, Grade: U

===== FACULTY DASHBOARD =====

1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit

5

===== MAIN MENU =====

1. Login as Admin
2. Login as Faculty
3. Login as Student
4. Exit

Enter your choice: 3

Enter Student Email: nandy@ssn.edu.in

Enter Student Password: Nandy@1234

Login successful!

===== STUDENT DASHBOARD =====

1. View Profile
2. View Grades
3. Apply for Revaluation
4. Exit

1

Profile: Student Name: Nandy, Email: nandy@ssn.edu.in, Department: cse, Joining Year: 2023, DOB: 04-11-2005, Gender: f

===== STUDENT DASHBOARD =====

1. View Profile
2. View Grades
3. Apply for Revaluation
4. Exit

2

Grades for Nandy:

os: U

Grades report updated in the file.

===== STUDENT DASHBOARD =====

1. View Profile
2. View Grades
3. Apply for Revaluation
4. Exit

```
===== STUDENT DASHBOARD =====
1. View Profile
2. View Grades
3. Apply for Revaluation
4. Exit
3
Enter the course for revaluation: os
Revaluation request submitted for course: os
===== STUDENT DASHBOARD =====
1. View Profile
2. View Grades
3. Apply for Revaluation
4. Exit
4
===== MAIN MENU =====
1. Login as Admin
2. Login as Faculty
3. Login as Student
4. Exit
Enter your choice: 2
Enter Faculty Email: spin@ssn.edu.in
Enter Faculty Password: Spin@1234
Login successful!
===== FACULTY DASHBOARD =====
```

===== FACULTY DASHBOARD =====

1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit

4

Enter student's email: nandy@ssn.edu.in

Revaluation request for course: os

Current Grade: 0

===== FACULTY DASHBOARD =====

1. View Profile
2. Set Grades for Students|
3. View Students' Grades
4. Review Revaluation Requests
5. Exit

2

Enter student's email: nandy@ssn.edu.in

Enter course name: [os](#)

Enter grade (O, A+, A, B+, B, C, U): [B](#)

Enter credit for course: [3](#)

Grade set successfully for Nandy

—
Enter student's email: nandy@ssn.edu.in

Enter course name: [os](#)

Enter grade (O, A+, A, B+, B, C, U): [B](#)

Enter credit for course: [3](#)

Grade set successfully for Nandy

===== FACULTY DASHBOARD =====|

1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit

[3](#)

Enter student's email: nandy@ssn.edu.in

Grades for Nandy:

Course: [os](#), Grade: [B](#)

===== FACULTY DASHBOARD =====

1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit

===== FACULTY DASHBOARD =====

1. View Profile
2. Set Grades for Students
3. View Students' Grades
4. Review Revaluation Requests
5. Exit

5

===== MAIN MENU =====

1. Login as Admin
2. Login as Faculty
3. Login as Student
4. Exit

Enter your choice: 4

Exiting the system!

8. Inference and Future Extensions

Inference:

This system effectively provides a structured approach to handling roles and data management for a college's grading system. By centralizing administrative, faculty, and student functions, it improves data consistency, security, and ease of access across different user roles within the institution.

Future Extensions:

- **Graphical User Interface (GUI):** Implementing a GUI would make the system more user-friendly and accessible, especially for users who are less familiar with command-line interactions.
- **Database Integration:** Moving from file-based storage to a database would enhance scalability, enabling efficient storage, retrieval, and management of large volumes of student and faculty data.
- **Parent Access:** Introducing a new user role for parents would allow them to view their child's grades and academic progress. This role could be designed with restricted access, focusing on read-only permissions for privacy and security.
- **Automated Grading Analytics:** Adding analytical tools could help generate insights into student performance trends, identify at-risk students, and support data-driven decisions by faculty and administrators.